LiveKit Load-Test Framework: Metrics Coverage Report

Scope: livekit-loadtest-tools (benchmark_livekit_calls.py + simple_livekit_audio_test.py). Basis: static review of the code. No benchmark has been run yet and no results file exists, so this report contains no measured results.

1. Summary

The framework measures how fast the API *accepts* call requests (HTTP latency, throughput, host CPU/memory). It does **not** measure call or media quality, nor LiveKit server health. Estimated single-machine capacity of the current code is roughly 50-150 concurrent calls (an estimate to be confirmed by a measured ramp test). Targets of 500 / 1,000 / 2,500 / 10,000 users need multiple instances or distributed generators (e.g. lk load-test).

2. Metrics currently covered

Metric	Status	Where / note
Total / successful / failed requests, success rate	Covered	benchmark_livekit_calls.py, CallResult / BenchmarkMetrics. Counts HTTP 200 on POST /calls only.
Requests per second (achieved)	Covered	total / duration. Includes ramp-down time, so slightly understated.
API latency: avg, min, max, p95, p99	Covered	Time for POST /calls to return. This is queue time, not call setup time.
Error breakdown by message	Covered	errors dict; HTTP status or exception text.
Load-generator CPU %, memory %, disk %, network MB	Covered	psutil, sampled every 1 s. Measures the generator host, not LiveKit.
Test shapes: sustained, burst, ramp-up	Covered	Three modes with configurable CPS / concurrency.
Per-call state: pending / active, progress	Partial	GET /calls/{id} exposes status, but the benchmark never polls it.
Memory profiling of the server	Partial	tracemalloc in server; not exported to the report, and adds overhead.
Result export	Partial	JSON only (--output). No charts, no PDF, no history comparison.

3. Metrics still needed

Metric	Status	Why it matters
Call setup time (request to participant connected)	Missing	True user-visible latency; the current latency is only API acceptance.
Call connect success rate (reached active / completed)	Missing	A 200 on POST /calls does not mean the call worked.
Concurrent active calls over time	Missing	Needed to prove 100 / 500 / 1000 / 2500 / 10000 were actually reached.
Audio frame pacing drift (actual vs 20 ms interval)	Missing	First thing to break under load in this design; shows generator saturation.
Packet loss, jitter, RTT, bitrate (per track)	Missing	Core media-quality metrics from LiveKit/WebRTC stats.
MOS / audio quality estimate	Missing	Standard voice-quality score for call tests.
Call duration and completion / drop rate	Missing	Detects calls dropped mid-playback.
LiveKit server CPU, memory, bandwidth, node count	Missing	Shows the server's real limit, not the generator's.
SIP metrics: INVITE success, post-dial delay, SIP error codes	Missing	Needed if testing phone calls via SIP trunk.
Room / participant counts on the server	Missing	Cross-check against intended load; also leaked rooms after test.
Error rate vs load (per ramp step), saturation point	Missing	Identifies the breaking point per tier.
Generator health: event-loop lag, open sockets, per-process CPU	Missing	Distinguishes generator limits from system-under-test limits.
Cleanup verification (rooms/calls removed)	Partial	clean_up_room.py exists but is not part of the run report.

4. Suggested targets per tier

Tier (concurrent users)	Approach	Pass criteria (suggested, adjust to SLA)
100	This repo, single machine, 1-2 processes	Connect success >= 99%, setup p95 < 3 s, frame drift p99 < 20 ms
500	This repo, 3-5 instances behind a script	Same as above; generator CPU < 70% per core
1,000 / 2,500	Distributed generators or lk load-test on several VMs	Server CPU < 70%, packet loss < 1%, jitter < 30 ms
10,000	Many generator nodes + multi-node LiveKit cluster (Redis)	No node saturation, error rate < 1% sustained 10 min

5. Next steps

• Run the ramp test at 10 / 25 / 50 / 100 calls and save the JSON output as the first real baseline.

• Add call-state polling, frame-drift tracking and concurrent-calls-over-time to the benchmark.

• Pull server-side metrics from LiveKit (Prometheus) during each tier.

• Generate this report automatically from the JSON results, with charts, once measured data exists.